# Portfolio – Expérience projet

Valentin De Araujo

## Sommaire

**01** Odomo — Station domotique en Java ..... 2
**02** Biosphere7 — Jeu de plateau & IA ..... 3
**03** Installation & administration de postes ..... 4
**04** Musicalertes — Création d'une base de données ..... 5
**05** Installation de services réseaux (Nextcloud) ..... 6
**06** Environnement réseau & administration système ..... 7
**07** Les Mystères de BUTopia — App C# POO ..... 8
**08** Retro-PC Dépannage — Refactorisation technique ..... 9
**09** Pixel War — Application JavaScript moderne ..... 10
**10** Coloration de graphes — Application Java ..... 11
**11** Blogue statique — Script Bash ..... 12
**12** Risk prevention in Nouvelle-Aquitaine ..... 13
**13** Capteurs — Analyse de températures en C ..... 14

# Odomo — Station domotique en Java

Programmation impérative · 4 semaines · Individuel

## Cahier des charges

- Développer une station domotique simulée en Java sous NetBeans.
- Gérer différents capteurs et actionneurs d'une maison (lumières, température, alarme…).
- Utiliser des tableaux, matrices et boucles comme seules structures de données.
- Concevoir des tests unitaires JUnit pour valider chaque fonctionnalité.

## Résultats obtenus

- Station fonctionnelle gérant plusieurs pièces simultanément.
- Tous les tests unitaires validés.
- Code structuré en méthodes claires, respectant les contraintes imposées (tableaux uniquement).

**Programme complet — tous les tests JUnit au vert**

## Méthodes et technologies

Java, NetBeans IDE, JUnit 5, Tableaux 2D, Boucles imbriquées, Méthodes statiques

## Compétences BUT exercées

- Réaliser (programmation Java impérative)
- Optimiser (algorithmes sur tableaux et boucles)
- Vérifier / Tester (JUnit)

## Commentaires

Apprentissage des bases de la logique algorithmique et de la syntaxe Java. C'est une fondation indispensable pour aborder sereinement la programmation orientée objet par la suite.

RET-EX 02 / 13 · SAÉ S1.01 / S1.02

# Biosphere7 — Jeu de plateau & IA

Algorithmique avancée · 8 semaines · Binôme

## Cahier des charges

- Développer un jeu de plateau sur une grille 14×14 simulant la compétition entre deux consortiums pour la gestion de plantes dans une biosphère martienne.
- Phase 1 (individuelle) : implémenter jusqu'à 16 niveaux de règles progressifs débloquant les suivants via l'interface fournie.
- Phase 2 (binôme) : coder deux intelligences artificielles en compétition sur le niveau final, avec analyse comparative des stratégies.

## Résultats obtenus

- 11 niveaux sur 16 validés avec succès.
- Deux IA aux stratégies distinctes codées (greedy vs heuristique de propagation).
- Interface graphique pour test fournie entièrement exploitable.

**11 niveaux + 2 IA complètes — Rapport stratégique rédigé**

## Méthodes et technologies

Java, Matrices 2D, Algorithmes gloutons, Heuristiques IA, Récursivité, Binôme / Git

## Compétences BUT exercées

- Réaliser (algorithmique en Java)
- Optimiser (stratégies et heuristiques d'IA)
- Collaborer (binôme)

## Commentaires

Découverte de la modélisation de problèmes complexes et de la création d'heuristiques. Savoir optimiser un algorithme sera un atout majeur pour développer des applications backend performantes.

# Installation & administration de postes

Systèmes d’exploitation · 2 semaines · Binôme

## Cahier des charges

- Simuler l’installation et la configuration d’un poste informatique via VMWare Workstation à partir d’un fichier ISO.
- Installer Debian 12 sur une machine virtuelle aux ressources très limitées (20 Go d’espace alloué).
- Créer et gérer les droits de deux utilisateurs : un compte administrateur (« alice ») et un compte développeur.
- Configurer l’environnement de développement (VS Code) et rédiger une procédure documentée reproductible, vérifiable par un tiers.

## Résultats obtenus

- Machine Debian 12 entièrement configurée, légère et fonctionnelle sous VMWare.
- Séparation stricte des privilèges validée entre l’administratrice et le développeur.
- Environnement de travail prêt à l’emploi, procédure étape par étape testée et validée.

**Environnement Debian optimisé (20 Go) avec séparation des rôles**

## Méthodes et technologies

VMWare Workstation, Debian 12 (ISO), VS Code, Bash / Terminal, Gestion utilisateurs Linux, Sudo / Droits

## Compétences BUT exercées

- Administrer (système Debian et machines virtuelles)
- Gérer les droits des utilisateurs (Linux)
- Documenter (procédure reproductible)

## Commentaires

Maîtrise de la virtualisation et de l’administration système sous Linux. Savoir configurer et gérer les droits d’un environnement de travail est essentiel pour tout poste de développeur ou DevOps.

# Musicalertes — Création d’une base de données

Modélisation & SQL · 4 semaines · Binôme

## Cahier des charges

- Concevoir from scratch la BD de la plateforme Musicalertes pour un service d’alertes concerts / festivals.
- Réaliser 5 étapes : recensement des données, description des services, MCD sous Win’Design, génération du MLR et script SQL DDL commenté.
- Modéliser les entités : artistes, concerts, festivals, lieux, clients et alertes SMS / email, puis fournir un script DML de remplissage.

## Résultats obtenus

- MCD validé et MLR généré puis enrichi manuellement.
- Scripts SQL exécutables sans erreur, incluant contraintes CHECK et commentaires détaillés.
- Jeu de données créé : 3 festivals, 10 concerts, 5 artistes, 8 clients.

**Scripts exécutés sans erreur — MCD approuvé**

## Méthodes et technologies

Win’Design, MCD / MLR, SQL DDL + DML, Clés étrangères, Contraintes CHECK, MySQL / MariaDB

## Compétences BUT exercées

- Gérer les données (base de données)
- Modéliser (du MCD au SQL)
- Documenter (scripts commentés)

## Commentaires

Assimilation du processus de modélisation de données (MCD / MLR) et de l’écriture de scripts SQL robustes. Une compétence clé, car la structuration des données est le cœur de toute application d’entreprise.

RET-EX 05 / 13 · SAÉ S2.03

# Installation de services réseaux (Nextcloud)

Réseaux & Systèmes · 2 semaines · Binôme

## Cahier des charges

- Mise en place d’une infrastructure réseau virtuelle complexe via NEmu avec deux sous-réseaux et une passerelle (routeur) assurant le routage inter-réseaux et l’accès Internet.
- Installation et configuration de Nextcloud pour le stockage et le partage de fichiers, couplé à OnlyOffice pour l’édition collaborative.
- Déploiement de clients lourds Nextcloud sur des postes clients et production d’une documentation technique (Wiki).

## Résultats obtenus

- Infrastructure réseau déployée avec succès sous NEmu.
- Services serveurs fonctionnels et sécurisés (HTTPS, 2FA, gestion des accès).
- Documentation technique complète (Wiki format Markdown) détaillant toutes les étapes.

**Réseau complet + Services serveur configurés — Wiki validé**

## Méthodes et technologies

NEmu, Debian 12, Nextcloud, OnlyOffice, Snap, HTTPS / TLS, Markdown

## Compétences BUT exercées

- Administrer (réseaux et systèmes)
- Installer et configurer des services
- Documenter (Wiki en Markdown)

## Commentaires

Mise en œuvre concrète d’une infrastructure réseau et de services de collaboration. Cette expérience me donne une vision globale du déploiement de services et de l’importance cruciale d’une documentation technique claire pour la maintenance.

RET-EX 06 / 13 · SAÉ S1.05

# Environnement réseau & administration système

Réseaux · Linux · 2 semaines · Binôme

## Cahier des charges

- Configurer un environnement réseau local complet en machines virtuelles : serveurs DHCP, DNS et HTTP, interfaces réseau.
- Gérer les utilisateurs et groupes Linux, avec scripting Bash pour l'automatisation de tâches d'administration.
- Rédiger une procédure technique détaillée pour permettre la reproduction de l'environnement, incluant des diagrammes réseau.

## Résultats obtenus

- Environnement réseau virtuellement déployé et entièrement opérationnel.
- Scripts d'automatisation Bash testés et validés en conditions réelles.
- Documentation complète livrée avec diagrammes réseau clairs et procédures étape par étape.

**Environnement reproductible — documentation approuvée**

## Méthodes et technologies

Bash scripting, Ubuntu Linux, DHCP / DNS, Réseau LAN, VMWare, Droits Unix (chmod)

## Compétences BUT exercées

- Administrer (réseau local)
- Automatiser (scripts Bash)
- Documenter (procédures et diagrammes réseau)

## Commentaires

Compréhension concrète de l'architecture d'un réseau local (DHCP, DNS) et de l'automatisation via des scripts Bash. Ces bases réseaux sont cruciales pour le déploiement et le diagnostic d'infrastructures.

# Les Mystères de BUTopia — App C# POO

Développement orienté objets · 4 semaines · Équipe de 3

## Cahier des charges

- Développer une application WinForms complète de gestion de fiches de personnages pour un jeu de rôles fictif (BUTopia).
- Implémenter des indicateurs numériques, des compétences critériées, un système d'épreuves avec jets de dés, un entraînement et une santé mentale évolutive.
- Assurer des livraisons de code en 4 stades, avec dossier UML complet (diagrammes de classes), tests NUnit et compte-rendus de projet.

## Résultats obtenus

- Application WinForms robuste intégrant création / édition de personnages, glisser-déposer et épreuves avec jets de dés.
- Système de gestion d'expérience implémenté et pleinement fonctionnel.
- Dossier UML fourni incluant un dictionnaire précis des responsabilités des classes.

**App fonctionnelle — 100 % tests verts — UML livré**

## Méthodes et technologies

C# / .NET, WinForms, NUnit, UML Diagrammes, Propriétés C#, Enum + extensions, Drag & Drop, Git

## Compétences BUT exercées

- Réaliser (programmation orientée objet en C#)
- Tester (NUnit)
- Collaborer (équipe de 3)
- Modéliser (UML)

## Commentaires

Acquisition des concepts de la programmation orientée objet (POO) et de la modélisation UML. Savoir concevoir une architecture logicielle propre et modulaire est indispensable pour coder des projets complexes en équipe.

# Retro-PC Dépannage — Refactorisation technique

Développement web moderne · 2 semaines · Individuel

## Cahier des charges

- Refactoriser complètement le site « Retro-PC Dépannage » avec des techniques web modernes sur 6 missions.
- Intégrer du HTML5 sémantique et des règles d’accessibilité strictes ; créer un Design System CSS (variables :root, police variable Google Fonts).
- Développer un layout CSS Grid + Flexbox en approche Mobile-First, avec transitions et animations @keyframes.
- Obtenir une validation W3C sans erreur et fournir un rapport technique réflexif.

## Résultats obtenus

- 3 pages HTML validées W3C sans aucune erreur résiduelle.
- Design system complet appliqué avec polices variables et animations fluides (fadeInUp, hover).
- Rapport réflexif exhaustif livré avec captures de la progression.

**W3C vert — Design system complet — Rapport livré**

## Méthodes et technologies

HTML5 sémantique, CSS Grid, Flexbox, Variables CSS, @keyframes, Media queries, W3C Validator

## Compétences BUT exercées

- Réaliser (développement web moderne)
- Concevoir un design system et soigner l’accessibilité
- Communiquer (rapport réflexif)

## Commentaires

Maîtrise de l’intégration web moderne (Grid, Flexbox, accessibilité) et de la refactorisation de code. Cette rigueur front-end me permettra de créer des interfaces professionnelles, maintenables et évolutives.

# Pixel War — Application JavaScript moderne

JavaScript · Fetch API · Canvas · 2 semaines · Individuel

## Cahier des charges

- Développer une application JavaScript moderne de type « Pixel War » (canvas collaboratif), avec outillage professionnel.
- Configurer un environnement Vite + npm pour le bundling et la gestion des dépendances.
- Implémenter la manipulation du Canvas HTML5 et des appels Fetch API vers un back-end JSON.
- Gérer l'authentification JWT via json-server-auth et structurer le code en modules ES6.

## Résultats obtenus

- Application Pixel War complète et collaborative en temps réel.
- Auth JWT fonctionnelle, canvas interactif avec détection de clics et coloration persistée via API REST.
- Build Vite final optimisé et prêt pour la production.

**App complète : auth + canvas + API REST — Build prod**

## Méthodes et technologies

JavaScript ES6+, Vite, npm, Canvas API, Fetch API, json-server-auth, JWT, Modules ES6, REST API

## Compétences BUT exercées

- Réaliser (JavaScript moderne)
- Intégrer une API et une authentification
- Utiliser un outillage moderne (Vite, npm)

## Commentaires

Prise en main de l'écosystème JavaScript professionnel (Vite, requêtes API REST, authentification JWT). Cette étape marque ma transition vers le développement web moderne, indispensable pour le développement fullstack.

# Coloration de graphes — Application Java

Développement orienté objet, graphes · 4 semaines · Binôme

## Cahier des charges

- Développer une application Java capable de **représenter visuellement un graphe**, de manipuler ses sommets et ses arêtes, d'appliquer différents algorithmes de coloration et de comparer leurs résultats.
- Implémenter la classe **Graph** (interface IGraph) : ajout et suppression de nœuds et d'arêtes, requêtes de voisinage, propriétés visuelles, à partir du code d'affichage fourni.
- Coder trois algorithmes de coloration : **Welsh-Powell**, **6-coloration** et **5-coloration** pour les graphes planaires.
- Concevoir des **tests unitaires JUnit**, puis **analyser la complexité** des méthodes et des algorithmes et mesurer leurs temps d'exécution.

## Résultats obtenus

- Classe Graph complète, développée méthode par méthode avec un test écrit après chaque méthode.
- Trois algorithmes de coloration fonctionnels : aucun voisin de même couleur, au plus 6 puis 5 couleurs sur les graphes planaires (testé jusqu'à une grille de 400 nœuds).
- 23 tests unitaires réussis : 11 sur les méthodes utilitaires, 2 sur Welsh-Powell, 6 sur la 6-coloration et 4 sur la 5-coloration.
- Complexité calculée (Welsh-Powell et 6-coloration en $O(V \cdot E)$, 5-coloration en $O(V^2 \cdot E)$) et vérifiée par des mesures de temps sur 5 graphes créés par nos soins.

**23 tests sur 23 réussis — 3 algorithmes de coloration complets — Analyse de complexité rédigée**

## Méthodes et technologies

Java, JUnit, UML, Javadoc, HashMap, Tri fusion, Récursivité, Graphes planaires, Complexité algorithmique

## Compétences BUT exercées

- Réaliser (développement orienté objet)
- Optimiser (algorithmes et complexité)
- Vérifier / Tester (JUnit)
- Modéliser (UML)
- Collaborer (binôme)

## Commentaires

Découverte de la théorie des graphes et de l'analyse de complexité sur un vrai projet Java. J'ai compris l'impact du choix d'une structure de données : sans liste d'adjacence, chaque requête de voisinage coûte O(E), ce qui limite les performances de tous les algorithmes. Un réflexe utile pour écrire du code efficace.

# Blogue statique — Script Bash

Scripting Bash · Markdown · Pandoc · 3 semaines · Binôme

## Cahier des charges

- Écrire un script Bash de gestion de blogue ou de site statique, appelé avec une commande et un paramètre optionnel (./blog.sh COMMANDE [PARAMETRE]).
- Gérer des pages écrites en Markdown : éditer, supprimer (avec confirmation) et lister les documents.
- Construire le site avec Pandoc : sommaire (index.md), une page HTML par document et un PDF unique (blog.pdf).
- Visualiser le résultat avec xdg-open et fournir une commande usage qui affiche la documentation du script.

## Résultats obtenus

- Script complet : cinq commandes (éditer, supprimer, lister, construire, visualiser) et la commande usage.
- Choix automatique de l'éditeur : variable d'environnement $EDITOR, puis /usr/bin/editor, puis recherche d'un éditeur présent sur la machine.
- Garde-fous : confirmation avant toute suppression et message clair « Il n'y a pas de fichier » quand le répertoire est vide.
- Construction du site : sommaire généré avec liens HTML, fichiers HTML dans web/ et PDF unique blog.pdf. Trace d'exécution fournie pour chaque commande.

**Script fonctionnel — 5 commandes + usage — Site HTML et PDF généré avec Pandoc**

## Méthodes et technologies

Bash, Pandoc, Markdown, xdg-open, Variables d'environnement ($EDITOR), Conditions Bash, Génération HTML / PDF

## Compétences BUT exercées

- Automatiser (Bash scripting)
- Réaliser (outil en ligne de commande)
- Documenter (usage, traces d'exécution)

## Commentaires

Les principales difficultés ont été les conditions et la syntaxe Bash, la commande build (enchaîner les étapes de génération avec Pandoc) et la gestion des erreurs. Ce projet m'a appris à automatiser une chaîne de publication complète en ligne de commande et à rendre un script plus sûr : confirmation, messages clairs et documentation intégrée.

RET-EX 12 / 13 · SAÉ S2.04 / S2.05

# Risk prevention in Nouvelle-Aquitaine

Exploitation de base de données · 2 semaines · Équipe de 6

## Cahier des charges

- Concevoir une application de visualisation et d'analyse des risques naturels en Nouvelle-Aquitaine (inondations, incendies de forêt, vagues de chaleur…) pour les habitants et le grand public, dans le cadre du défi national OpenDataUniversity (data.gouv.fr).
- S'appuyer sur trois sources Open Data : Gaspar (catastrophes naturelles et plans de prévention des risques), Météo-France (données météo quotidiennes) et Atmo Nouvelle-Aquitaine (qualité de l'air).
- Rédiger le dossier de spécifications : lexique, fonctions priorisées, contraintes, structure du logiciel, modèle conceptuel de la base de données et caractérisation de l'interface.
- Réaliser une analyse statistique (régression linéaire partielle, histogrammes) avec des données intégrées en fixe dans une interface graphique ; architecture en trois couches : interface C# (.NET 8.0), moteur Python (Pandas, Matplotlib, Seaborn, GeoPandas) et base de données.

## Résultats obtenus

- Dossier de spécifications rédigé à 6 : fonctions priorisées (doit avoir, devrait avoir, peut avoir, n'aura pas), modèle conceptuel de la base de données et caractérisation de l'interface.
- Base de données regroupant les trois sources Open Data, avec environ 4 millions de données traitées en Python.
- Application en trois couches (interface C#, moteur Python, base de données) permettant de croiser risques naturels, météo et qualité de l'air.

**Application en 3 couches — 3 sources Open Data — environ 4 millions de données traitées en Python**

## Méthodes et technologies

C# (.NET 8.0), Visual Studio, Python, Pandas, Matplotlib, Seaborn, GeoPandas, SQL, SQLite, Régression linéaire, Données Open Data

## Compétences BUT exercées

- Gérer les données (base de données et Open Data)
- Réaliser (interface C# et traitement Python)
- Collaborer (équipe de 6)
- Documenter (dossier de spécifications)

## Commentaires

Projet de grande ampleur : une équipe de 6, trois technologies (C#, Python, SQL) et un volume de données de l'ordre de 4 millions. J'ai appris à nettoyer et traiter de gros jeux de données avec Python, à relier plusieurs langages dans une même application et à travailler avec un cahier des charges priorisé pour répartir le travail.

# Capteurs — Analyse de températures en C

Développement C & Système · 2 semaines · Individuel

## Cahier des charges

- Construire un programme C analysant les températures issues d'un historique de mesures JSON récupéré via un flux HTTP.
- Implémenter des structures C, des tableaux dynamiques et gérer rigoureusement l'allocation mémoire (malloc, realloc, free).
- Calculer des statistiques de température (minimum, maximum, moyenne, écart) à partir des données traitées.
- Générer un rapport formaté dans le terminal et exporter une synthèse CSV, avec gestion des arguments en ligne de commande.

## Résultats obtenus

- Programme C compilé via Makefile et structuré en plusieurs fichiers sources et headers.
- Gestion de la mémoire dynamique validée par Valgrind, sans aucune fuite ni erreur de segmentation.
- Interface en ligne de commande (CLI) opérationnelle avec parsing des options (--url, --file, --csv, --label).
- Tests unitaires, tests de rapports et analyse statique avec cppcheck validés avec succès.

**Application C robuste — 100 % tests verts — Aucune fuite mémoire (Valgrind)**

## Méthodes et technologies

C, GCC, Makefile, Valgrind, cppcheck, VS Code, Pointeurs, Allocation dynamique, CLI

## Compétences BUT exercées

- Réaliser (développement système en C, gestion d'arguments CLI)
- Optimiser (allocation de la mémoire dynamique, structures et pointeurs)
- Vérifier / Tester (Make tests, Valgrind, analyse statique cppcheck)

## Commentaires

Ce projet m'a permis de consolider mes bases en programmation C, avec un focus particulier sur la sécurité et la gestion de la mémoire manuelle. La rigueur imposée par des outils professionnels comme Valgrind et cppcheck m'a appris à concevoir un code système robuste, sans fuites ni erreurs de segmentation.